Plan de pruebas de la historia de usuario Ver detalle de estación (id: 532236)

CRITERIOS DE ACEPTACIÓN

Prueba 00. Éxito:

1. El técnico selecciona una estación de la lista de estaciones de una red.
2. La aplicación muestra los detalles de la estación seleccionada. Esto es: el ID, el nombre, la latitud y longitud, la hora de última actualización, el número de bicicletas disponibles y de huecos libres.
3. Se verifica que los datos mostrados de la estación son correctos.

Prueba 01. Datos ausentes:

Nota: Esto quiere decir que alguno de los datos de la estación no está disponible en el servicio.

1. El técnico selecciona una estación de la lista de estaciones de una red.
2. La aplicación detecta que hay datos de la estación ausentes.
3. La aplicación muestra los datos disponibles de la estación.
4. Se verifica que los datos mostrados de la estación son correctos.
5. Se verifica que para los datos no disponibles de la estación se muestra un guion (indicando que el dato no está disponible).

Prueba 02. Datos anómalos:

Nota: Esto quiere decir bien que la latitud (-90, 90) o la longitud (-180, 180) están fuera del rango permitido, o bien que el número de bicicletas disponibles o de huecos libres es negativo.

1. El técnico selecciona una estación de la lista de estaciones de una red.
2. La aplicación detecta que hay datos de la estación anómalos.
3. La aplicación muestra los datos normales de la estación.
4. Se verifica que los datos mostrados de la estación son correctos.
5. Se verifica que para los datos anómalos de la estación se muestra un guion (indicando que el dato no es válido).

Prueba 03. Servicio de datos no disponible:

1. El técnico selecciona una estación de la lista de estaciones de una red.
2. La aplicación detecta que el servicio de datos no responde.

3. Se verifica que la aplicación notifica al técnico que ha habido un error interno.

Prueba 04. Error al comunicarse con el servicio de datos:

1. El técnico selecciona una estación de la lista de estaciones de una red.
2. La aplicación detecta que ha ocurrido un error al acceder al servicio de datos.
3. Se verifica que la aplicación notifica al técnico que ha habido un error interno.

Prueba 05. Conexión de red no disponible:

1. El técnico selecciona una estación de la lista de estaciones de una red.
2. La aplicación detecta que el dispositivo del técnico no dispone de conexión a la red.
3. Se verifica que la aplicación notifica al técnico que el dispositivo no dispone de conexión a internet.

PRUEBAS DE INTERFAZ DE USUARIO

Prueba 00. Éxito:

1. El técnico selecciona la estación ”377 - Metro Abrantes” de la lista de estaciones de la red BiciMAD.
2. La aplicación muestra los detalles de la estación seleccionada. Esto es:
    - El ID con “01f5011f76069b1e81a11d4d51dd9d1d”.
    - El nombre con ”377 - Metro Abrantes”.
    - La latitud y longitud con los valores “40.38091762, -3.72783615”.
    - La hora de última actualización siendo “2026-09-24T11:11:40.490774+00:00Z”.
    - El número de bicicletas disponibles siendo 0.
    - Los huecos libres son 18.
3. Se verifica que los datos mostrados de la estación son correctos.

Prueba 01. Datos ausentes:

Nota: Esto quiere decir que alguno de los datos de la estación no está disponible en el servicio.

1. El técnico selecciona la estación "390 - Estacion datos ausentes" de la lista de estaciones de la red BiciMAD.
2. La aplicación detecta que hay datos de la estación ausentes y muestra los datos normales junto con un guion ("-") en los ausentes:
    - ID con “01f5011f76069b1e81a11d4d51dd9d1d”.
    - El nombre es "390 - Estacion datos ausentes"
    - Latitud y longitud con “40.38091762, -3.72783615”
    - La hora de última actualización que debe ser “-”

- Bicicletas disponibles con “-”
- Los huecos libres son “-”

3. Se verifica que los datos normales mostrados son correctos y que para los datos ausentes o no disponibles (hora de última actualización, bicicletas disponibles y huecos libres) se muestra un guion ("-").

Prueba 02. Datos anómalos:

Nota: Esto quiere decir bien que la latitud (-90, 90) o la longitud (-180, 180) están fuera del rango permitido, o bien que el número de bicicletas disponibles o de huecos libres es negativo.

1. El técnico selecciona la estación "377 - Metro Abrantes" de la lista de estaciones de la red BiciMAD teniendo datos anómalos (latitud 95.0, longitud -185.0, bicicletas -1).
2. La aplicación detecta que hay datos de la estación anómalos y muestra los datos normales junto con un guion ("-") en los anómalos:
   - ID con “01f5011f76069b1e81a11d4d51dd9d1d”.
   - El nombre es "377 - Metro Abrantes"
   - Latitud y longitud con “-, -”
   - La hora de última actualización que debe ser “2026-09-24T11:11:40.490774+00:00Z”
   - Bicicletas disponibles con “-”
   - Los huecos libres son “-”
3. Se verifica que los datos normales mostrados son correctos y que para los datos anómalos (latitud, longitud, bicicletas disponibles y huecos libres) se muestra un guion ("-").

Prueba 03. Servicio de datos no disponible:

1. El técnico selecciona la estación "377 - Metro Abrantes" de la lista de estaciones de la red BiciMAD estando el servicio de datos inactivo.
2. La aplicación detecta que el servicio de datos no responde.
3. Se verifica que la aplicación notifica al técnico que ha habido un error interno.

Prueba 04. Error al comunicarse con el servicio de datos:

1. El técnico selecciona la estación "377 - Metro Abrantes" de la lista de estaciones de la red BiciMAD produciéndose un fallo al acceder al servicio de datos
2. La aplicación detecta que ha ocurrido un error al acceder al servicio de datos.
3. Se verifica que la aplicación notifica al técnico que ha habido un error interno.

Prueba 05. Conexión de red no disponible:

1. Con la conexión a Internet del dispositivo desactivada, el técnico selecciona la estación "377 - Metro Abrantes" de la lista de estaciones de la red BiciMAD.

2. La aplicación detecta que el dispositivo del técnico no dispone de conexión a la red.
3. Se verifica que la aplicación notifica al técnico que el dispositivo no dispone de conexión a internet.

METODOS SUSCEPTIBLES DE PRUEBA UNITARIO O INTEGRACION

1. onCreate(): Método encargado de solicitar los detalles de la estación seleccionada a la capa de negocio/persistencia, validar que los datos estén disponibles y dentro de los rangos válidos (latitud en (-90, 90), longitud en (-180, 180), bicicletas disponibles >= 0 y huecos libres >= 0), sustituir por un guion ("-") aquellos datos ausentes o anómalos, y ordenar a la vista mostrar la información o notificar un error interno.

2. Presenter.init() (Ver Anexo): Método genérico de lógica de presentación encargado de validar los datos del modelo y ordenar a la vista mostrarlos o sustituirlos por guiones. Susceptible de **prueba unitaria** mediante Mocks aislados.

PRUEBAS UNITARIAS

Durante el diseño e implementación de esta historia de usuario, el equipo decidió prescindir de la capa de presentación (Presenter) para la vista de detalle de estación (StationDetailsView), dado que la pantalla actúa únicamente como un receptor pasivo de un objeto Station previamente descargado y empaquetado en el Intent desde la vista anterior. Al encontrarse la lógica de validación y formateo de campos acoplada directamente en el método del ciclo de vida de Android StationDetailsView.onCreate(), no existen unidades funcionales aisladas del framework susceptibles de prueba unitaria pura en esta historia.

Siguiendo las indicaciones de la profesora durante la sesión de validación y seguimiento del plan de pruebas:

1. La verificación del comportamiento de StationDetailsView.onCreate() se certifica mediante Pruebas de Integración (StationDetailsViewITest) utilizando Robolectric y Mockito.
2. Con el fin de cubrir los objetivos de evaluación individual relativos a la especificación, codificación y ejecución de pruebas unitarias aisladas con uso de mocks, se ha acordado documentar e implementar en el anexo de este documento las pruebas unitarias correspondientes al método RedDetailsPresenter.init() (implementadas en la clase RedDetailsPresenterTest).

PRUEBAS DE INTEGRACION

Id de prueba	Entrada	Salida esperada
Prueba 00.	Se crea un Mock de Station con Mockito que, al invocar sus *getters*, devuelve los datos de la estación "377 - Metro Abrantes" (id = "01f5011f76069b1e81a11d4d51dd9d1d", latitude = 40.38091762, longitude = -3.72783615, timestamp = "2026-09-	La Actividad extrae correctamente el Mock del Intent. La aserción verifica que el TextView de bicicletas disponibles muestra "0" y los demás campos muestran sus

	24T11:11:40.490774+00:00Z", freeBikes = 0, emptySlots = 18). El Mock se empaqueta en el Intent de inicio.	valores exactos, sin invocar el mensaje de error interno.
Prueba 01.	Se configura un Mock de Station que devuelve los nombres de "390 - Estacion datos ausentes" y la longitud y la latitud “40.38091762, -3.72783615” además debe de ser null al consultar la hora de actualización, bicicletas y huecos.	La lógica del onCreate evalúa correctamente los nulos provenientes del Mock y asigna la cadena de recurso R.string.noData ("-") a los TextView correspondientes, validando la integración del manejo de ausencia de datos.
Prueba 02.	Se configura un Mock de Station que devuelve valores fuera del rango geográfico y lógico: latitude = 95.0, longitude = -185.0, freeBikes = -1.	El onCreate integra los datos del Mock, las validaciones lógicas detectan los valores anómalos (> 90, < -180, < 0) y la vista asigna R.string.noData ("-") exclusivamente a esos campos, manteniendo los válidos (como el nombre o ID).
Prueba 03.	Se lanza la Actividad con un Intent que no contiene ningún extra INTENT_STATION (simulando un fallo previo en la red).	El onCreate detecta que el objeto extraído es null y ejecuta la rama de error, mostrando el Toast con el mensaje R.string.loadErrorStation.

RESULTADOS DE LAS PRUEBAS

Responsables de los artefactos de prueba:

Autor del Plan de Pruebas: David García.

Autor y ejecutor de las pruebas (Unitarias, Integración y UI): David García.

Responsable de corrección de defectos en el código de producción: Aarón Alegría (implementador de la historia de usuario).

Nivel de Prueba	Clase de Prueba	Caso de Prueba	Resultado Inicial	Resultado Final (Post-corrección)
Integración	StationDetailsViewITest	IT-00 (Datos válidos)	Éxito (PASS)	Éxito (PASS)
Integración	StationDetailsViewITest	IT-01 (Datos	Fallo (FAIL)	Éxito (PASS)

		ausentes / nulos)		
Integración	StationDetailsViewITest	IT-02 (Datos anómalos)	Fallo (FAIL)	Éxito (PASS)
Integración	StationDetailsViewITest	IT-03 (Sin extra / Error de carga)	Éxito (PASS)	Éxito (PASS)
Unitaria (Anexo)	RedDetailsPresenterTest	UT-01, UT-02, UT-03	Éxito (PASS)	Éxito (PASS)
Interfaz (UI)	VerDetalleEstacionExito00UITest	Prueba 00. Éxito	Éxito (PASS)	Éxito (PASS)
Interfaz (UI)	VerDetallesEstacionDatoAusente01UITest	Prueba 01. Datos ausentes	Éxito (PASS)	Éxito (PASS)

Ejecución de Pruebas de Integración (IT-01 e IT-02): Durante la primera ejecución de StationDetailsViewITest, los casos de prueba IT-01 (datos ausentes) e IT-02 (datos anómalos) resultaron fallidos. En IT-01, al configurar el Mock de Station para devolver null en atributos numéricos (hora, bicicletas disponibles y huecos libres), se producía un error (NullPointerException) debido a que el modelo Station utilizaba tipos de datos primitivos (double e int) que no admiten valores nulos. Asimismo, en IT-02, las condiciones de validación en StationDetailsView.onCreate() no filtraban correctamente la totalidad de los rangos anómalos (valores negativos o fuera de coordenadas geográficas), mostrando el valor erróneo en lugar del guion ("-").

Ejecución de Pruebas Unitarias (RedDetailsPresenterTest): En la ejecución de los casos UT-01, UT-02 y UT-03 del método init() de los detalles de la red no se detectaron fallos, cumpliendo la cobertura de caminos prevista a la primera.

ANEXO: ESPECIFICACION DE PRUEBAS UNITARIAS DEL METODO INIT() (RedDetailsPresenter)

Dado que el método init() del RedDetailsPresenter delega en métodos estáticos para comprobar la conectividad y la ubicación, se especifican a continuación las pruebas unitarias siguiendo técnicas de caja blanca (cobertura de caminos) y aislando sus dependencias estáticas mediante Mocks (Mockito).

A.1. Clases de Equivalencia y Caminos Lógicos

- **Conexión a Red (Utils.isNetworkAvailable):**
  - **CE1 (Válida):** Devuelve true. Se evalúa la ubicación y se procede a cargar los datos.
  - **CE2 (Inválida):** Devuelve false. Se notifica error de red y se aborta el flujo.

- **Ubicación (Utils.isLocationEnabled):**
    - **CE3 (Válida):** Devuelve true. No se muestran errores y se carga la red.
    - **CE4 (Inválida):** Devuelve false. Se notifica el error de ubicación pero se continúa con la carga de la red.

A.2. Casos de Prueba Unitarios

Caso de pruebas	Entrada	Salida esperada
UT-01	NetworkAvailable = true LocationAvailable = true	Se debe invocar view.init() y ejecutar el método interno de carga llamando a repository.requestNetwork(). No se debe invocar ningún mensaje de error.
UT-02	NetworkAvailable = true LocationAvailable = false	Se invoca view.init() y detecta la falta de GPS invocando view.showLocationError(). Posteriormente, continúa el flujo y ejecuta repository.requestNetwork().
UT-03	NetworkAvailable = false LocationEnabled = (No se evalúa)	La lógica invoca view.init() y detecta la falta de conexión, invocando view.showRedError(). La ejecución se detiene y **nunca** se llama a repository.requestNetwork().